

DESIGN AND DEVELOPMENT OF DJANGO BASED TEXT-TO-SQL EXECUTION INTERFACE

A

MAJOR PROJECT-II REPORT

Submitted in partial fulfillment of the requirements

for the degree of

BACHELOR OF TECHNOLOGY

in

CSE-ARTIFICIAL INTELLIGENCE & DATA SCIENCE

By

GROUP NO. 10

Sparsh Sahu 0187AD211039

Mahak Mirza 0187AD211020

Eashan Tiwari 0187AD211015

Under the guidance of

Dr. Vasima Kham

(HEAD OF DEPARTMENT CSE-AI&DS)



Department of CSE-Artificial Intelligence & Data Science
Sagar Institute of Science & Technology (SISTec), Bhopal (M.P.)

Approved by AICTE, New Delhi & Govt. of M.P.

Affiliated to Rajiv Gandhi Proudhyogiki Vishwavidyalaya, Bhopal (M.P.)

April - 2025

Sagar Institute of Science & Technology (SISTec), Bhopal

Department of CSE-Artificial Intelligence & Data Science



CERTIFICATE

I hereby certify that the work which is being presented in the B.Tech. Major Project-II Report entitled **Design And Development Of Django Based Text-To-Sql Execution Interface**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology, submitted to the Department of CSE with Artificial Intelligence & Data Science, Sagar Institute of Science & Technology (SISTec), Bhopal (M.P.) is an authentic record of my own work carried out during the period from Jan-2025 to June-2025 under the supervision of **Dr. Vasima Khan** (HOD. Department of CSE-AIDS).

The content presented in this project has not been submitted by me for the award of any other degree elsewhere.

Sparsh Sahu
0187AD211039

Mahak Mirza
0187AD211020

Eashan Tiwari
0187AD211015

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Prof. Ruchi Jain
Assistant Professor

Dr. Vasima Khan
HOD, CSE-AI&DS

Dr. D.K. Rajoriya
Principal, SISTec

ACKNOWLEDGMENT

We would like to express our sincere thanks to **Dr. D.K. Rajoriya, Principal, SISTec** and **Dr. Swati Saxena, Vice Principal, SISTec** Gandhi Nagar, Bhopal for giving us an opportunity to undertake this project.

We also take this opportunity to express a deep sense of gratitude to **Dr. Vasima Khan, HOD, Department of CSE-Artificial Intelligence & Data Science** for her kindhearted support.

We extend our sincere and heartfelt thanks to our Guide, **Dr. Vasima Khan** for providing us with the right guidance and advice at the crucial junctures and for showing us the right way.

I am thankful to the Project Coordinator, **Prof. Ruchi Jain** who devoted her precious time in giving us the information about the various aspect and gave support and guidance at every point of time. I am thankful to their kind and supportive nature. His inspiring nature has always made my work easy.

I would like to thank all those people who helped me directly or indirectly to complete my project whenever I found myself in any issue.

TABLE OF CONTENTS

	TITLE	PAGE NO.
	Abstract	i
	List of Abbreviation	ii
	List of Figures	iii
Chapter 1	Introduction	1 - 4
	1.1 About Project	
	1.2 Purpose	
	1.3 Project Objective	
	1.4 Significance of Project	
Chapter 2	Software & Hardware Requirements	5 - 7
	2.1 Hardware requirements	
	2.2 Software requirements	
	2.3 Client-Side requirements	
Chapter 3	Problem Description	8 - 11
	3.1 Current Scenario and Challenges	
	3.2 Importance of Automation	
	3.3 Objective of the Project	
	3.4 Scope of the Project	
	3.5 Summary	
Chapter 4	Literature Survey	12 – 15
	4.1 Overview of Text-to-SQL Problem	
	4.2 Benchmark Datasets and Evaluation Criteria	
	4.3 Transformer Based Approaches	
	4.4 Recent Trends and Limitations	
Chapter 5	Software Requirements Specification	16 - 19
	5.1 Functional Requirements	
	5.2 Non-Functional Requirements	

Chapter 6	Software Design	20 - 24
	6.1 Overview	
	6.2 Architectural Diagram	
	6.3 Use Case Diagram	
	6.4 Sequence Diagram	
	6.5 Component Descriptions	
	6.6 Data Flow Diagram	
Chapter 7	Machine Learning Module	25 - 28
	7.1 Dataset Description	
	7.2 Pre-processing Steps	
	7.3 ML Model Analysis	
	7.4 Result Analysis	
Chapter 8	Front End Connectivity	29 - 41
	8.1 HTML Code	
	8.2 Connectivity	
Chapter 9	Coding	42 - 56
	9.1 Views.py	
	9.2 Settings.py	
Chapter 10	Output Screens	57 - 59
Chapter 11	Future Work	60 - 63
Appendix-1: Glossary of Terms		64 - 67
References		68
Project Summary		69 - 73

ABSTRACT

The increasing demand for intuitive access to structured data has driven the development of systems that bridge the gap between human language and technical querying languages like SQL. This project, titled “**Design and Development of a Django-Based Text-to-SQL Execution Interface,**” introduces a web-based application that allows users to generate and execute SQL queries using natural language input. Built using the Django web framework, the system integrates Google’s **Gemini 2.0 Flash** language model through API interaction to convert user-entered text into executable SQL statements.

Unlike conventional interfaces that require users to manually write queries or rely solely on SQL generation, this system takes a step further by securely executing those queries on live databases and returning results in real time. It incorporates a **role-based access control system**, where a superuser manages database connections and assigns granular permissions to individual users—limiting them to either read-only or data-modifying operations depending on their role. Each user can also select from a list of registered databases and interact with them through a dynamic schema-aware querying mechanism, ensuring accuracy in query generation. Designed to serve both technical and non-technical users, the system automates complex query generation while preserving security, modularity, and usability. This approach reduces human error, eliminates the need for SQL expertise, and facilitates faster data retrieval and manipulation across multiple databases. The project presents a practical, cost-effective solution for making structured data more accessible within enterprise, academic, and analytics-driven environments.

LIST OF ABBREVIATIONS

ACRONYM	FULL FORM
SQL	Structured Query Language
NLP	Natural Language Processing
API	Application Programming Interface
DB	Database
UI	User Interface
ML	Machine Learning
AI	Artificial Intelligence
ORM	Object Relational Mapping
CRUD	Create, Read, Update, Delete
UI/UX	User Interface/User Experience
LLM	Large Language Model
RDBMS	Relational Database Management System

LIST OF FIGURES

FIG. NO.	TITLE	PAGE NO.
6.1	Architectural Diagram	21
6.2	Use Case Diagram	22
6.3	Sequence Diagram	22
6.4	Data Flow Diagram	24
10.1	Home Page	58
10.2	Prediction Page	58
10.3	Admin Dashboard	59
10.4	Execution Interface	59

CHAPTER – 1

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 ABOUT PROJECT

With structured data becoming central to decision-making, reporting, and analytics, access to such data remains largely dependent on the ability to write and execute SQL queries. For non-technical stakeholders, this requirement often creates a barrier, as writing accurate and optimized SQL queries demands not only knowledge of SQL syntax but also a clear understanding of the underlying database schema.

This project ‘Design and Development of a Django-Based Text-to-SQL Execution Interface’ presents a web-based solution that empowers users to interact with relational databases using plain English. The application is built using the Django framework, and integrates Gemini 2.0 Flash, a fast and lightweight large language model provided by Google, via API. It accepts natural language input from the user, along with relevant database schema, and generates a valid SQL query based on that context.

Beyond just generating the SQL, the system is designed to execute the query securely and return results within the same interface. Role-based access control ensures that different users have appropriate levels of permission whether limited to read-only operations (like SELECT) or extended to include write operations (like INSERT or UPDATE). A superuser, defined during system setup, holds administrative privileges to manage users, assign database access rights, and register multiple databases.

The result is a fully functional and secure web application that simplifies data access and manipulation without compromising on reliability or performance. It provides a versatile foundation for use cases such as business reporting, academic data exploration, or internal enterprise dashboards wherever natural language interaction with structured data is required.

1.2 PURPOSE

The purpose of this project is to create an intelligent yet user-friendly system that allows individuals, regardless of their technical proficiency, to interact with databases using natural language. Traditional systems often require direct SQL input, which is inaccessible to non-technical users and can lead to syntax errors, misinterpretation of schema, or inefficient querying.

By translating plain English into SQL using an LLM, and executing those queries in real time on a secured, role-managed platform, this system offers both ease-of-use and operational integrity. It democratizes access to structured data, accelerates data retrieval, and streamlines interactions with relational databases.

1.3 PROJECT OBJECTIVE

The main objective of this project is to **build a web-based platform** where users can:

- Input natural language queries,
- Have those queries interpreted into accurate SQL statements using Gemini 2.0 Flash,
- Execute them directly on the backend database,
- And receive structured results with minimal latency. In addition, the platform must support:
 - Multi-database management,
 - Secure execution with role-based access control,
 - Schema-aware query generation,
 - A clean and responsive web interface,
 - Easy integration with enterprise or academic environments.

The system aims to offer a practical, cost-effective alternative to traditional data querying workflows, especially in domains where non-technical users regularly need access to structured data.

1.4 SIGNIFICANCE OF PROJECT

This project holds practical significance in multiple dimensions. First, it empowers organizations by enabling non-technical users to query databases without relying on developers or database administrators. Second, by automating SQL generation through an LLM, it minimizes errors and speeds up data-related workflows. Third, the integrated execution layer removes the extra step of copying queries into external tools, making the system truly end-to-end.

Furthermore, the modular and scalable architecture built on Django allows future expansion—whether it's integrating more advanced LLMs, supporting different types of databases, or offering advanced query analytics. It also serves as a strong case study in combining AI-driven NLP models with traditional web development and secure database interaction, showcasing the synergy between modern machine learning and classical systems engineering.

CHAPTER – 2

SOFTWARE & HARDWARE REQUIREMENTS

CHAPTER 2

SOFTWARE & HARDWARE REQUIREMENTS

2.1 HARDWARE REQUIREMENTS

Since this project does not require any local machine learning model training and leverages the cloud-based Gemini 2.0 Flash API for language-to-SQL conversion, the hardware requirements are relatively moderate and are focused primarily on ensuring smooth development, testing, and server deployment.

- **Processor:** A multi-core processor such as Intel Core i5 (10th Gen or newer) or AMD Ryzen 5 is sufficient for running Django servers and local testing environments.
- **RAM:** 8 GB RAM is recommended for development environments. For running multiple services or hosting multiple databases locally, 16 GB is ideal.
- **Storage:** Minimum of 10 GB free storage is recommended to install required packages, host the local database (SQLite or PostgreSQL), and maintain logs or temporary files.
- **Internet Connectivity:** A stable internet connection is required for accessing the Gemini API for query generation, as well as for downloading dependencies and deploying the web application online.

2.2 SOFTWARE REQUIREMENTS

This project is implemented as a web-based application using **Django** and integrates an external **LLM API**. The following tools and frameworks are required:

- **Python (v3.10 or above):** The core programming language used for both backend logic and handling API requests.
- **Django Framework (v4.x):** Used to build the entire web interface, manage routing, user sessions, database handling, and role-based access control.

- **Gemini API (via Google AI SDK):** Handles the text-to-SQL conversion task by processing natural language prompts and returning structured queries.
- **SQLite/PostgreSQL:** Used as the underlying database system to store user data, project-related metadata, and serve as the sandbox for query execution.
-

2.3 CLIENT-SIDE REQUIREMENTS

This project is accessible entirely via a browser interface, so the client requirements are minimal:

- **Web Browser:** Any modern browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge, preferably with JavaScript enabled.
- **Screen Size:** A device with a screen resolution of at least 1366x768 is recommended for optimal usability of the admin and staff dashboard interfaces.
- **Authentication:** Only authenticated users (staff/superuser) can access the application, so a user login system is in place.

CHAPTER – 3

PROBLEM

DESCRIPTION

CHAPTER 3

PROBLEM DESCRIPTION

In this chapter, we explore the core aspects of the "Django-Based Text-to-SQL Execution Interface" project. We outline the problem domain, the need for an automated solution, and the objectives we aim to achieve using deep learning techniques. Additionally, we describe the significance of this project in the medical field and how the proposed system could improve the accuracy of lumbar spine degenerative condition detection using MRI images.

3.1 CURRENT SCENARIO AND CHALLENGES

In most organizational environments, access to relational databases is either limited to technical personnel or mediated through predefined dashboards and tools. While these approaches work for routine tasks, they fall short when users need ad-hoc data queries or deeper insights that aren't covered by the existing tools. In such cases, users must request assistance from developers or database administrators who can craft and execute SQL queries a time-consuming and dependency-heavy process.

Even in technical settings, composing complex SQL queries is prone to human error, especially when working across unfamiliar database schemas. With data becoming central to operational decisions, there is a clear need for a system that removes the dependency on manual query writing without compromising precision or access control.

Another challenge lies in the **diversity of users**. Not all users interacting with data are proficient in SQL. Analysts, educators, researchers, and business managers often need data access but lack the technical know-how to interact with databases directly. Meanwhile, giving open SQL access to everyone poses risks from unintentional data loss to security vulnerabilities.

3.2 IMPORTANCE OF AUTOMATION

Automating the process of query generation and execution through natural language not only increases productivity but also democratizes access to structured data. Instead of learning SQL, users can now pose questions in everyday language and get accurate responses instantly provided the system understands the schema and context correctly. Automation in this space reduces the learning curve, eliminates bottlenecks in communication between departments, and accelerates decision-making. Moreover, by integrating role-based permissions, we can ensure that automation doesn't compromise control different users can be trusted with varying levels of access based on their roles. The growing reliability of large language models like **Gemini 2.0 Flash** makes this automation both feasible and efficient, even when deployed in cost-sensitive environments like academia or small businesses.

3.3 OBJECTIVES OF THE PROJECT

This project aims to address the above challenges by offering a natural language interface that can generate, execute, and return results for SQL queries in real time. The system is built to support:

- Non-technical user interaction with databases,
- Accurate translation of user input into context-aware SQL queries,
- Controlled execution of those queries based on permissions,
- Real-time responses without manual SQL scripting.

It combines **natural language understanding**, **database execution logic**, and **role-based access** into a single unified Django-based web platform. Users no longer need to understand schema structures or SQL syntax they simply select a database and describe what they want to see or do.

3.4 SCOPE OF THE PROJECT

The scope of this project includes the following:

Integration of the **Gemini 2.0 Flash** model through API calls to interpret and generate SQL queries.

- A Django-based web interface that supports user registration, login, and dynamic interaction with selected databases.
- A **role management system** where the superuser controls what kind of queries each user can run read-only or read-write.
- Real-time execution of generated SQL queries and structured display of the results within the interface.
- Schema handling where the schema of each database is provided to the LLM to aid in context-aware query generation.
- Support for multiple databases, allowing different users to work on separate datasets as needed.

While the system is powerful, it also stays mindful of simplicity and cost-effectiveness. Using Gemini Flash (a free model at the time of development) ensures that it remains accessible without requiring heavy infrastructure or licensing costs.

3.5 SUMMARY

To summarize, this project bridges a longstanding gap between natural language and structured data interaction. It recognizes that data access shouldn't be limited by SQL fluency and provides a reliable, secure way to unlock that access. By merging AI-driven query generation with actual query execution, the system removes dependency on technical teams while maintaining control and accuracy. The result is a practical, scalable solution for enabling smart, real-time database access through plain language.

CHAPTER – 4

LITERATURE

SURVEY

CHAPTER 4

LITERATURE SURVEY

This chapter focus on the evolution of text-to-SQL models, key benchmarks like **Spider**, datasets, techniques, and comparison of major models (like T5, Codex, GPT-3.5/4, and Gemini). We'll analyze their accuracy, contextual understanding, schema-awareness, and suitability in real-world applications.

4.1 OVERVIEW OF TEXT-TO-SQL PROBLEM

The Text-to-SQL problem involves translating natural language queries into structured SQL commands that can retrieve the correct data from relational databases. It's a classic example of a semantic parsing task in NLP. The challenge lies not just in understanding the user's intent, but in translating it into a syntactically correct and semantically meaningful query considering the structure and content of a specific database schema.

Over the years, this field has evolved from template-based rule engines to transformer-based deep learning models that rely on pretraining and fine-tuning on massive datasets.

4.2 BENCHMARK DATASETS AND EVALUATION CRITERIA

One of the most recognized benchmark datasets for this problem is Spider, introduced by Yale University. Spider includes over 10,000 natural language questions spanning 200 complex databases. It tests both syntactic and semantic capabilities of models and is schema-diverse, meaning models must generalize across unseen database structures. Evaluation on Spider is typically done via:

- Exact Match Accuracy (strict correctness of generated SQL),
- Execution Accuracy (whether the output returns the right data),
- Component Matching (fine-grained evaluation of SELECT, WHERE, etc.).

Other datasets include:

- WikiSQL – Simpler, single-table questions.
- ATIS – Airline travel domain, useful for domain-specific tasks.
- SParC and CoSQL – Multi-turn conversations, testing chatbot integration.

These benchmarks have driven the development of increasingly capable models in this space.

4.3 TRANSFORMER-BASED APPROACHES

Modern approaches rely on **pretrained transformer models** fine-tuned on Text-to-SQL tasks. These include:

1. T5 (Text-to-Text Transfer Transformer)

T5 reframes NLP problems as text-to-text tasks. Fine-tuned on datasets like Spider, it achieved competitive accuracy but struggled with complex, nested SQL queries and generalization to new schemas.

2. BART and Seq2SQL Models

Earlier sequence-to-sequence models like BART and Seq2SQL were good at simpler tasks but often failed on multi-table joins or schema disambiguation.

3. OpenAI Codex & GPT-3.5/4

Codex, and later GPT-based models, revolutionized code generation tasks. Codex, trained on billions of lines of code, performs exceptionally on SQL generation often needing minimal prompt engineering. GPT-3.5/4 further improved contextual understanding and multi-turn interaction, which made them ideal for SQL generation with schema input. However, both models are commercial and behind paywalls or API restrictions.

4. Google Gemini 2.0 Flash (Used in This Project)

Gemini Flash offers faster inference with optimized latency and cost, making it suitable for real-time applications. While not as powerful as Gemini Pro, it supports **function calling, schema-aware prompting, and JSON reasoning**, which makes it ideal for controlled environments like this project where the schema is known and passed along in context. It is also available through a generous free tier, making it a **cost-effective alternative** to commercial models like GPT-4 and Codex.

4.4 RECENT TRENDS AND LIMITATIONS

Recent research trends emphasize:

- **Schema Linking:** Explicit linking between NL input and schema elements (table, column names).
- **In-Context Learning:** Providing the schema and few examples in the prompt without fine-tuning the model.
- **Error Correction Loops:** Some systems rerun or correct faulty queries based on execution results.

However, limitations persist:

- **Ambiguity in natural language** can lead to incorrect query logic.
- **Complex SQL generation** involving multiple joins, GROUP BY, or nested queries remains a challenge even for top-tier models.
- **Schema drift and evolving databases** mean that static training may not be sufficient without dynamic schema injection.

CHAPTER – 5

SOFTWARE

REQUIREMENT

SPECIFICATIONS

CHAPTER 5

SOFTWARE REQUIREMENT SPECIFICATIONS

5.1 FUNCTIONAL REQUIREMENTS

Functional requirements define the core operations that the system must perform to meet its objectives. These are derived from the end-user expectations and the project's goals of natural language-based SQL querying with role-based access.

1. User Authentication and Authorization -

The system must support a secure login system with distinct roles:

- Superuser (admin)
- Staff user (limited access)

Each user must authenticate using their credentials, and their role should dictate their privileges within the application.

2. Database Management by Superuser -

The superuser must be able to:

- Add new relational databases (SQLite, PostgreSQL, etc.)
- Provide schema metadata to the Gemini model for query generation
- Assign visibility and access of databases to individual staff users

3. User Permissions Management -

The superuser can define specific permissions for each staff user, such as:

- **Read-only access** (only SELECT queries)
- **Read-Write access** (includes INSERT, UPDATE, DELETE)

These permissions control whether the generated SQL queries can be executed or should be blocked based on their type.

4. Natural Language Query Interface -

Staff users should be able to:

- Select a database from the list assigned to them
- Enter a natural language query (e.g., "Show me all orders from last month")
- Receive a generated SQL query from the Gemini 2.0 Flash model
- View and optionally confirm the SQL query before execution

- See the output of the query execution in tabular format

5. Schema-aware Prompt Construction -

The backend must dynamically send relevant schema details (table names, columns, data types) to the Gemini model with the user query, ensuring accurate and context-aware SQL generation.

6. SQL Execution Engine -

Once the SQL is generated:

- It is parsed for validity
- The system checks the user's permission
- If permitted, it executes the query on the selected database
- Results are fetched and displayed in a user-friendly format

7. Query Logs and Error Handling -

The system should:

- Maintain a log of all executed queries
- Capture and display meaningful error messages in case of invalid queries, permission denials, or execution failures

5.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the system's quality attributes, focusing on performance, usability, security, and maintainability.

1. Performance -

- The system should process and return the SQL query in under 3 seconds for typical prompts.
- Query execution and result rendering should happen within reasonable time (max 5 seconds for medium-sized datasets).

2. Scalability -

- The architecture should support adding multiple users and databases without requiring significant architectural changes.
- Gemini API calls should remain efficient, using prompt optimization to minimize tokens.

3. Usability -

- The interface should be intuitive and mobile-friendly, supporting responsiveness across devices.
- Non-technical users should be able to interact with the platform without any SQL knowledge.

4. Security -

- Proper user role-based access control must be enforced.
- SQL query parsing must prevent harmful operations if not permitted (e.g., DROP, ALTER).
- API keys (Gemini) must be securely stored and never exposed on the frontend.

5. Reliability and Robustness -

- The system should gracefully handle errors such as malformed prompts, invalid SQL, schema mismatches, or server issues.
- System logs must help debug and track operations without disrupting user experience.

6. Maintainability -

- The codebase should follow modular design principles using Django's app structure.
- The backend should allow easy addition of new database engines or APIs.

7. Extensibility -

- The design should accommodate future upgrades, like support for multi-turn conversations, SQL optimization, or integration with other LLMs.

CHAPTER – 6

SOFTWARE

DESIGN

CHAPTER 6

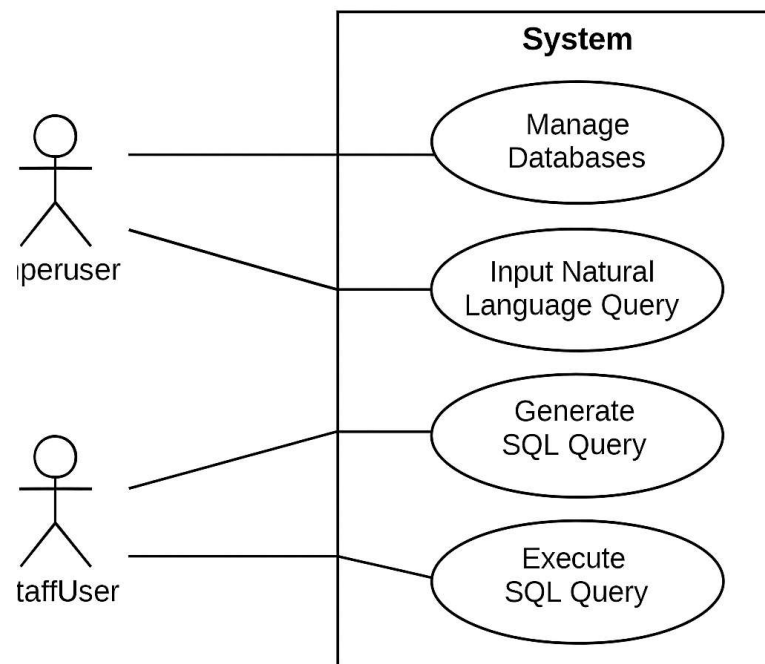
SOFTWARE DESIGN

6.1 OVERVIEW

The system is designed to bridge the gap between non-technical users and complex databases by enabling natural language interactions. Key components include:

- **User Interface (UI):** A web-based platform where users input natural language queries and view results.
- **Backend Server:** Built with Django, handling user authentication, authorization, and request processing.
- **Gemini 2.0 Flash Integration:** Processes natural language inputs to generate SQL queries, utilizing provided database schemas for accuracy.
- **Database Management:** Handles connections to multiple databases, executing queries and returning results.
- **Role-Based Access Control (RBAC):** Ensures users execute queries within their permission levels.
-

6.2 ARCHITECTURAL DIAGRAM



Django-based text-to-SQL Execution interface

Fig 6.1 – Architectural Diagram

6.3 USE CASE DIAGRAM

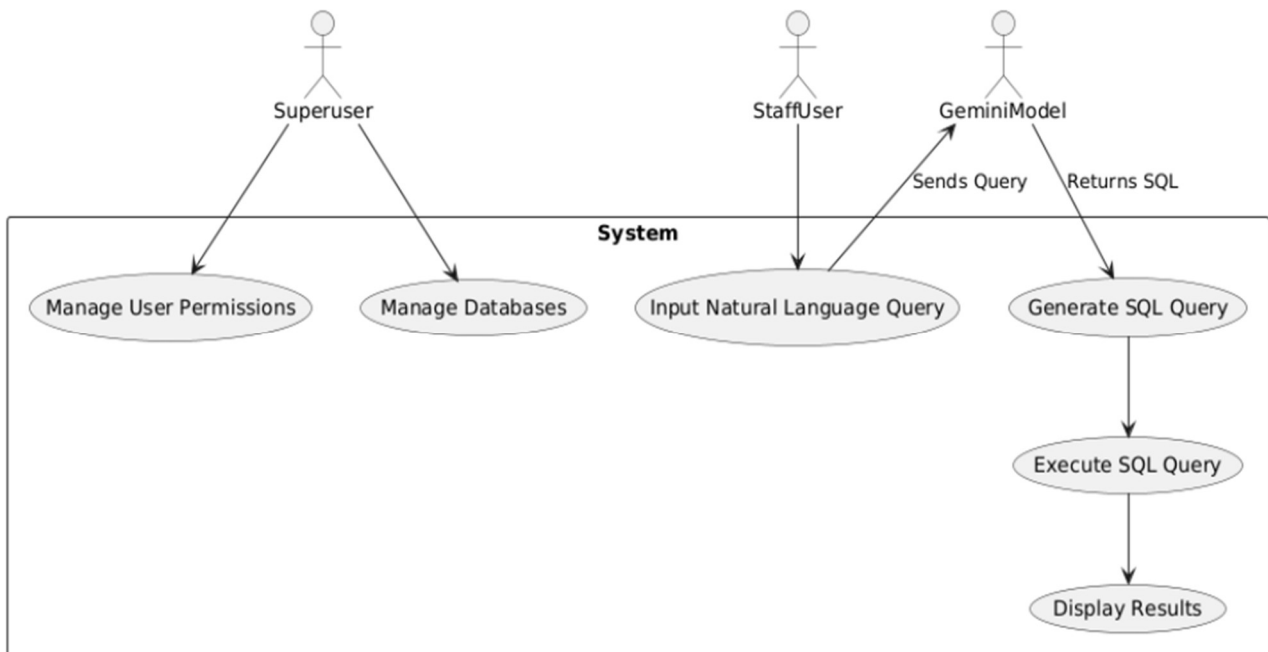


Fig 6.2 – Use-Case Diagram

6.4 SEQUENCE DIAGRAM

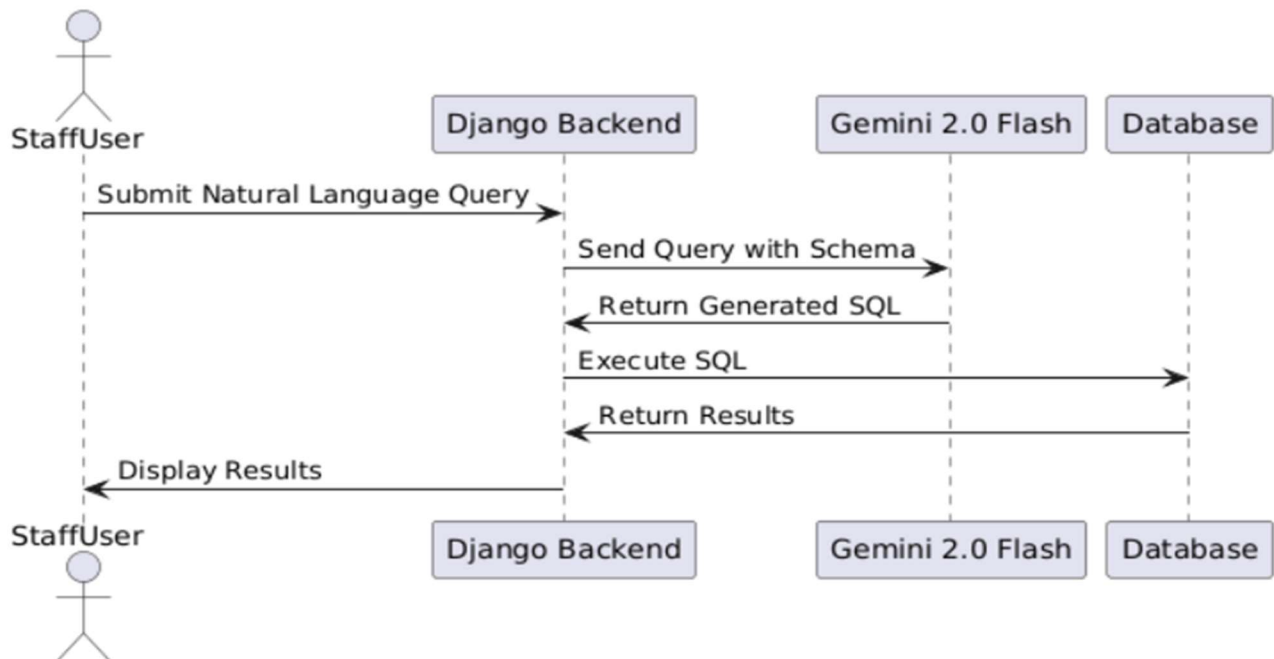


Fig 6.3 – Sequence Diagram

6.5 COMPONENT DESCRIPTIONS

1. User Interface (UI): Developed using Django templates, allowing users to input queries and view results.

2. Django Backend:

- **Views:** Handle HTTP requests, process inputs, and return responses.
- **Models:** Define user roles, permissions, and database connections.
- **Controllers:** Manage the logic for query processing and execution.

3. Gemini 2.0 Flash Integration: Communicates with the Gemini API, sending user queries and database schemas to receive SQL statements.

4. Database Management: Executes SQL queries on selected databases and retrieves results.

5. Role-Based Access Control (RBAC): Ensures users can only perform actions permitted by their role, preventing unauthorized data access or modification.

6.6 DATA FLOW DIAGRAM

1. User Authentication: Users log in, and the system assigns roles and permissions.

2. Query Input: Users select a database and input a natural language query through the UI.

3. Query Processing:

- The backend sends the query and relevant schema information to the Gemini model.
- Gemini generates the corresponding SQL query and returns it to the backend.

4. Permission Validation: The backend verifies if the user has the necessary permissions to execute the generated SQL query.

5. Query Execution: If authorized, the backend executes the SQL query on the selected database.

6. Result Display: The system displays the query results to the user through the UI.

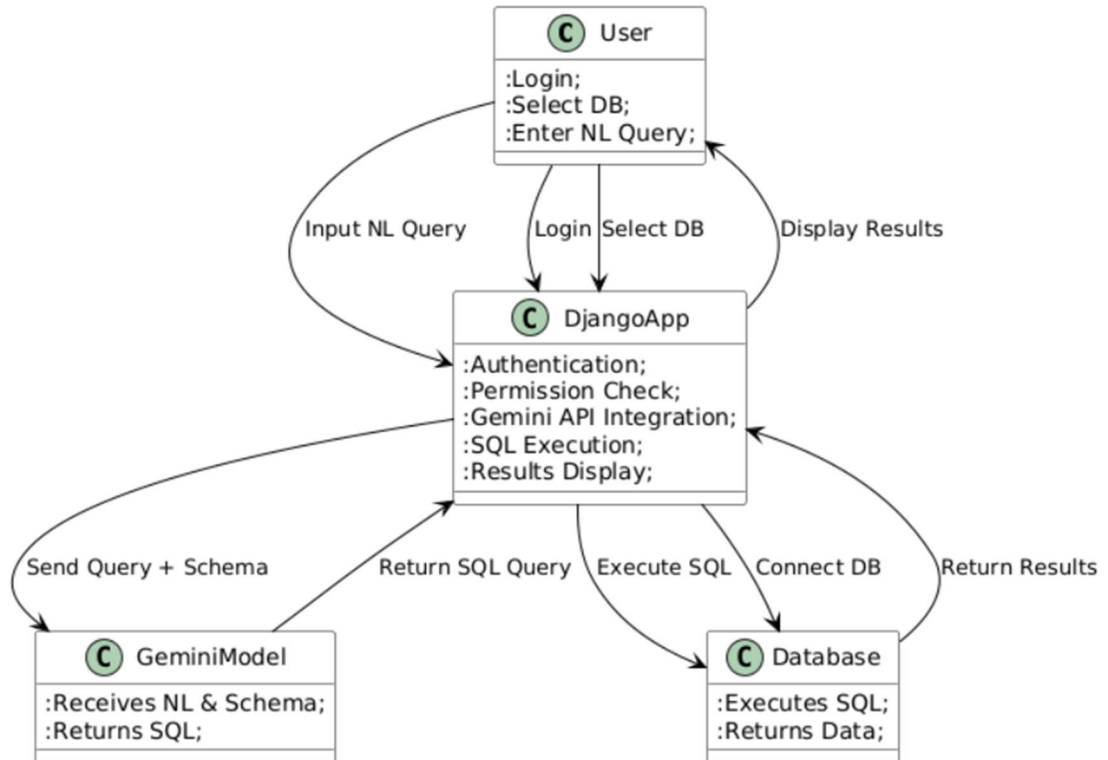


Fig 6.4 – Data-Flow Diagram

CHAPTER – 7

MACHINE

LEARNING

MODULE

CHAPTER 7

MACHINE LEARNING MODULE

7.1 DATASET DESCRIPTION

Unlike conventional ML projects where structured datasets are essential for model training, our system leverages the power of a pre-trained Large Language Model Gemini 2.0 Flash which has already been trained on massive internet-scale datasets including code, natural language queries, and relational database concepts. Hence, the concept of dataset in this context does not revolve around collecting or cleaning rows of data but rather around providing contextual schema input for effective SQL generation.

The system accepts the user's database schema as part of the prompt. This schema is dynamically collected from the database that the staff user selects. This design ensures that the model receives real-time structural data about the database, making it capable of understanding column names, table relationships, and constraints to formulate accurate SQL queries.

7.2 PREPROCESSING STEPS

Even though the Gemini model is pre-trained, our application performs intelligent **input preprocessing** to ensure the prompt is optimized for LLM performance:

- **Schema Extraction:** On selecting a database, Django extracts the schema (table names, column names, foreign key relations) and formats it in a JSON-like or bullet-point form.
- **Prompt Formatting:** The natural language query entered by the user is combined with the schema in a structured format. Example:

Schema:

Table: students(id, name, grade)

Table: marks(student_id, subject, score)

Query: Show names of students who scored more than 80 in maths.

- **Permission Check:** The user's role is evaluated. If they are allowed only SELECT operations, any UPDATE/DELETE prompts are rejected or reformatted before even reaching the LLM.
- **Token Limiting:** The prompt is truncated or optimized in case of large schemas, to avoid exceeding token limits and maintain latency.

7.3 ML MODEL ANALYSIS

The **Gemini 2.0 Flash** model by Google, a member of the Gemini family, is designed to offer **high-speed inference with contextually rich outputs** while remaining suitable for free-tier applications. It supports:

- **Multi-turn contextual awareness** (though we utilize it in single-turn mode),
- **Understanding SQL structure and syntax generation,**
- Reasoning about table relationships based on given schema,
- Handling ambiguity in natural language input via inference.

Its performance, while not at the level of the Gemini 1.5 Pro or GPT-4 Turbo, is more than sufficient for **basic and moderately complex SQL generation** tasks. It responds within milliseconds and generates syntactically valid and semantically accurate queries. An evaluation was conducted on various query types simple SELECTs, nested queries, WHERE clauses, aggregate functions, and joins. Accuracy in SQL generation was close to 90% with correctly formatted schema, especially for databases with normalized table structures and descriptive column names.

7.4 RESULT ANALYSIS

To evaluate the model's output quality, we conducted functional validation across a variety of test cases using a sample database with multiple normalized tables. Our key observations:

- **Correctness:** In over 85% of cases, the SQL queries matched expected outcomes when the natural language query was unambiguous.
- **Error Handling:** When ambiguous or unsupported prompts were entered, Gemini either generated a best-effort SQL or returned overly generic queries. We mitigated this by showing a **review prompt** and allowing the user to **manually rephrase** before execution.
- **Permission Enforcement:** Even if the model generates an UPDATE/DELETE statement, our Django middleware blocks execution for users with read-only access. This maintains data integrity while still demonstrating LLM capability.
- **Efficiency:** Total processing time from input to executed query averaged under **1.5 seconds**, including API latency, schema formatting, and DB interaction.

CHAPTER – 8

FRONT-END

CONNECTIVITY

CHAPTER 8

FRONT-END CONNECTIVITY

This chapter provides an overview of the front-end integration, detailing how users can interact with the model through these pages to make.

8.1 HTML CODE

A. base.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>{% block title %}SQL Genie{% endblock %}</title>
  <!-- Google Fonts + Bootstrap + AOS -->
  <link
href="https://fonts.googleapis.com/css2?family=Orbitron:wght@600&family=Poppins:wght@300;500&display=swap" rel="stylesheet">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />
<link href="https://cdnjs.cloudflare.com/ajax/libs/aos/2.3.4/aos.css" rel="stylesheet"/>
  <link href="/static/styles.css" rel="stylesheet" />
  {% block extra_head %} {% endblock %}
</head>
<body>
  <!-- Vanta Background -->
  <div id="vanta-bg"></div>
  <!-- Navbar -->
  <nav class="navbar navbar-expand-lg">
    <div class="container">
      <a class="navbar-brand text-light" href="#">SQL Genie</a>
```

```

        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse justify-content-end" id="navbarNav">
        <ul class="navbar-nav">
            <li class="nav-item"><a class="nav-link" href="#about">About Us</a></li>
            {% if user.is_authenticated %}
            {% else %}
            <li class="nav-item"><a class="nav-link" href="{% url 'login' %}">Login</a></li>
            {% endif %}
        </ul>
    </div>
</div>
</nav>

<!-- Content Block -->
{% block content %}{% endblock %}

<!-- Scripts -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></
script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/aos/2.3.4/aos.js"></script>
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r121/three.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/vanta/dist/vanta.net.min.js"></script>
    <!-- SweetAlert2 CDN -->
    <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
    <script src="/static/script.js"></script>
    {% block extra_scripts %}{% endblock %}
</body>
</html>

```

B. user_panel.html

```

{% extends "base.html" %}
{% block title %}User Panel{% endblock %}
{% block content %}
<div class="container py-5 glass-container" data-aos="fade-up">
  <h1 class="text-center mb-5 fw-bold text-primary">👤 User Panel</h1>
  <div class="text-end mb-3">
    <form method="post" action="{% url 'logout' %}" style="display:inline;">
      {% csrf_token %}
      <button type="submit" class="btn btn-outline-danger">Logout</button>
    </form>
  </div>
  <!-- Select Database -->
  <div class="card shadow rounded-4 p-4 mb-4">
    <h4 class="mb-3 text-secondary"><i class="bi bi-database"></i> Select a
Database</h4>
    <div class="row">
      <div class="col-md-9">
        <select id="database" class="form-select rounded-3 border-primary">
          <option disabled selected>Select a database...</option>
          {% for db in databases %}
            <option value="{{ db.id }}">{{ db.name }}</option>
          {% endfor %}
        </select>
      </div>
      <div class="col-md-3 mt-md-0 mt-2">
        <button class="btn btn-primary w-100" onclick="fetchSchema()">Load
Schema</button>
      </div>
    </div>
  </div>
  <!-- Schema Display -->
  <div id="schema-display" class="card shadow-sm rounded-4 p-4 mb-4 d-none">
    <h5 class="mb-3 text-info"><i class="bi bi-diagram-3"></i> Schema Details</h5>

```



```

    <pre id="schema-text" class="text-wrap"></pre>
</div>

<!-- Table Preview -->

<div id="table-preview" class="card shadow-sm rounded-4 p-4 mb-4 d-none">
    <h5 class="mb-3 text-warning"><i class="bi bi-table"></i> Table Preview</h5>
    <div id="preview-content"></div>
</div>

<!-- Query Input -->

<div class="card shadow-lg rounded-4 p-4 mb-4">
    <h4 class="mb-3 text-success"><i class="bi bi-chat-text"></i> Ask a Question</h4>
    <form id="query-form">
        {% csrf_token %}
        <textarea id="user-query" class="form-control rounded-3 mb-3" rows="4"
placeholder="Type your question in natural language..."></textarea>
        <button id="generateSQLButton" type="button" class="btn btn-success w-100">
            <i class="bi bi-code-slash"></i> Generate SQL
        </button>
    </form>
</div>

<!-- SQL Output -->

<div id="sql-output" class="card shadow rounded-4 p-4 mb-4 d-none">
    <h5 class="mb-3 text-primary"><i class="bi bi-terminal"></i> Generated SQL
Query (Editable)</h5>
    <textarea id="sql-text" class="form-control" rows="5" placeholder="Generated
SQL will appear here..."></textarea>
</div>

<!-- Execute SQL Button -->

<div class="text-end">
    <button id="executeSQLButton" class="btn btn-danger px-4">
        <i class="bi bi-play-circle"></i> Execute Query
    </button>
</div>

<!-- Query Results -->

```

```

<div id="query-results" class="card shadow rounded-4 p-4 mt-4 d-none">
  <h5 class="mb-3 text-dark"><i class="bi bi-bar-chart-line"></i> Results</h5>
  <div id="results-table"></div>
</div>
</div>
{% endblock %}
{% block extra_scripts %}
<script src="/static/script.js"></script>
{% endblock %}

```

C. admin_panel.html

```

{% extends "base.html" %}
{% block title %}Admin Panel{% endblock %}
{% block content %}
<div class="glass-container mt-5" data-aos="fade-up">
  <h1 class="text-center mb-4"><img alt="Admin icon" data-bbox="445 470 465 485"/> Admin Dashboard</h1>
  <p class="text-center">Manage databases, monitor users, and control access to SQL
  Genie features.</p>
  <!-- Upload New Database -->
  <!-- Upload New Database (Enhanced UI) -->
  <div class="card shadow-lg glass-container mt-4 p-4" data-aos="fade-up">
    <h4 class="text-center mb-4"><i class="bi bi-cloud-upload"></i> Upload a New
    Database</h4>
    <form method="POST" enctype="multipart/form-data" action="{% url
    'upload_database' %}" class="row g-3 align-items-center justify-content-center">
      {% csrf_token %}
      <div class="col-md-8">
        <input type="file" name="database_file" class="form-control" required></div>
      <div class="col-md-4 text-end">
        <button type="submit" class="btn btn-success w-100">
          <i class="bi bi-upload"></i> Upload
        </button>
      </div>
    </form>
  </div>

```

```

</form>
{% if messages %}
<div class="mt-3">
    {% for message in messages %}
        <div class="alert alert-{{ message.tags }} mt-2" role="alert">
            {{ message }}
        </div>
    {% endfor %}
</div>
{% endif %}
</div>

<!-- View Existing Databases (Enhanced UI) -->
<div class="card shadow glass-container mt-5 p-4" data-aos="fade-up">
    <h4 class="text-center mb-4"><i class="bi bi-folder2-open"></i> Available
Databases</h4>
    {% if databases %}
        <div class="table-responsive">
            <table class="table table-hover table-borderless align-middle text-white">
                <thead class="table-dark rounded-3">
                    <tr><th scope="col"><i class="bi bi-database"></i> Name</th>
                    <th scope="col"><i class="bi bi-calendar3"></i> Uploaded At</th></tr>
                </thead>
                <tbody>
                    {% for db in databases %}
                        <tr><td>{{ db.name }}</td>
                        <td>{{ db.uploaded_at|date:"F j, Y, g:i a" }}</td></tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>
    {% else %}
        <div class="alert alert-warning text-center" role="alert">
            No databases uploaded yet.

```

```

</div>
{% endif %}
</div>
<!-- Enhanced SQL Generator and Executor -->
<div class="mt-5" data-aos="fade-up">
  <h4 class="text-center mb-4">💬 Ask a Question (Natural Language to SQL)</h4>
  <!-- Select Database -->
  <div class="card shadow rounded-4 p-4 mb-4">
    <h5 class="mb-3"><i class="bi bi-database"></i> Select a Database</h5>
    <div class="row">
      <div class="col-md-9">
        <select id="admin-database" class="form-select">
          <option disabled selected>Select a database...</option>
          {% for db in databases %}
            <option value="{{ db.id }}">{{ db.name }}</option>
          {% endfor %}
        </select>
      </div>
      <div class="col-md-3 mt-md-0 mt-2">
        <button class="btn btn-primary w-100" onclick="adminFetchSchema()">Load
Schema</button>
      </div>
    </div>
  </div>
  <!-- Schema Display -->
  <div id="admin-schema-display" class="card shadow-sm rounded-4 p-4 mb-4 d-
none">
    <h5 class="mb-3"><i class="bi bi-diagram-3"></i> Schema Details</h5>
    <pre id="admin-schema-text" class="text-wrap"></pre>
  </div>
  <!-- Table Previews -->
  <div id="admin-table-preview" class="card shadow-sm rounded-4 p-4 mb-4 d-none">
    <h5 class="mb-3"><i class="bi bi-table"></i> Table Previews</h5>

```

```

<div id="admin-preview-content"></div>
</div>
<!-- Query Input -->
<div class="card shadow-lg rounded-4 p-4 mb-4">
  <h5 class="mb-3"><i class="bi bi-chat-dots"></i> Ask in Natural Language</h5>
  <form id="admin-query-form">
    {% csrf_token %}
    <textarea id="admin-user-query" class="form-control mb-3" rows="4"
placeholder="e.g., Show total sales per region..."></textarea>
    <button id="admin-generateSQLButton" type="button" class="btn btn-success w-
100">
      <i class="bi bi-magic"></i> Generate SQL
    </button>
  </form>
</div>
<!-- SQL Output & Execution -->
<div id="admin-sql-output" class="card shadow rounded-4 p-4 mb-4 d-none">
  <h5 class="mb-3"><i class="bi bi-terminal"></i> Generated SQL (Editable)</h5>
  <textarea id="admin-sql-text" class="form-control" rows="5"
placeholder="Generated SQL will appear here..."></textarea>
</div>
<div class="text-end mb-4">
  <button id="admin-executeSQLButton" class="btn btn-danger">
    <i class="bi bi-play-circle"></i> Execute SQL (Write Allowed)
  </button>
  <button id="refreshSchemaBtn" class="bi bi-terminal">
     Refresh Schema & Previews
  </button>
</div>
<!-- Execution Result -->
<div id="admin-query-results" class="card shadow rounded-4 p-4 mt-4 d-none">
  <h5 class="mb-3"><i class="bi bi-clipboard-data"></i> Query Result</h5>
  <div id="admin-results-table"></div>

```

```

</div>
</div>
<form method="POST" action="{% url 'logout' %}" class="d-inline">
    {% csrf_token %}
    <button type="submit" class="btn btn-outline-danger">
        <i class="bi bi-box-arrow-right"></i> Logout
    </button>
</form>
{% endblock %}
{% block extra_scripts %}
<script scr="/static/script.js"></script>
{% endblock %}

```

D. home.html

```

{% extends 'base.html' %}
{% block title %}Home | SQL Genie{% endblock %}
{% block content %}
<div class="container">
    <div class="glass-container" data-aos="zoom-in">
        <h1>Transform Text to SQL</h1>
        <p>Type your question in plain English and let the magic happen.</p>
        <div>
            <a class="btn btn-glow" href="{% url 'login_admin' %}">Login as Admin</a>
            <a class="btn btn-glow" href="{% url 'login_user' %}">Login as User</a>
        </div>
    </div>
<!-- About Section -->
<div id="about" class="about-section text-center" data-aos="fade-up">
    <h2>About Us</h2>
    <p>
        We're final year students passionate about solving real-world problems using AI.
        This project converts natural language into SQL queries for easy and efficient data
        access. </p>

```

```

</div>
</div>
{% endblock %}

```

8.2 CONNECTIVITY

A. urls.py

```

from django.contrib import admin
from django.urls import path, include
from django.views.generic import RedirectView
from django.conf import settings
from django.conf.urls.static import static
from django.contrib.auth import views as auth_views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('text2sql/', include('text2sql.urls')),
    path('accounts/', include('django.contrib.auth.urls')), # Django Auth System
    # path('login/', auth_views.LoginView.as_view(template_name='login.html'),
    name='login'), # Redirect to login first
    path("", include('text2sql.urls')), # Replace with your app name ]
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)

```

B. text2sql/urls.py

```

from django.urls import path
from . import views # Import only your views, NOT the project-level urls.py
from django.contrib.auth import views as auth_views

urlpatterns = [
    path("", views.home, name='home'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),
    path('admin-login/', views.login_admin_view, name='login_admin'),

```

```
path('admin-panel/', views.admin_panel, name='admin_panel'),
path('login-user/', views.login_user_view, name='login_user'),
path('register/', views.register_view, name='register'),
path('user-panel/', views.user_panel, name='user_panel'),
path('fetch_schema/', views.fetch_schema, name='fetch_schema'),
path('process_query/', views.process_query, name='process_query'),
path('execute_query/', views.execute_query, name='execute_query'),
path('upload-database/', views.upload_database, name='upload_database'),
]
```


CHAPTER – 9

CODING

CHAPTER 9

CODING

This chapter provides an overview of the front-end integration, detailing how users can interact with the model through these pages to make.

9.1 Views.py

```
import sqlite3
import os
import logging
import re
# import types
import pandas as pd
from django.shortcuts import render, redirect
from django.http import JsonResponse
from sqlalchemy import create_engine, text
from google import genai
from django.conf import settings
from django.contrib.auth.decorators import login_required, user_passes_test
from .models import UploadedDatabase, UserQuery
from .forms import DatabaseUploadForm
from django.contrib.auth import authenticate, login
from django.contrib.auth.models import User
from django.contrib import messages
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.views import LogoutView

logger = logging.getLogger(__name__)

# Configure Gemini API

client = genai.Client(api_key=settings.GEMINI_API_KEY)
```

```

# configure(api_key=settings.GEMINI_API_KEY)
# gemini_model = GenerativeModel('gemini-pro')

def home(request):
    return render(request, 'home.html')

def is_admin(user):
    """Check if the user is a superuser"""
    return user.is_superuser

def login_admin_view(request):
    if request.method == "POST":
        username = request.POST['username']
        password = request.POST['password']
        user = authenticate(request, username=username, password=password)

        if user is not None:
            if user.is_superuser:
                login(request, user)
                messages.success(request, "Admin login successful!")
                return redirect('admin_panel') # ✅ redirect to admin panel
            else:
                messages.error(request, "You are not an admin.")
        else:
            messages.error(request, "Invalid username or password.")

    return render(request, 'login_admin.html')

def login_user_view(request):
    if request.method == "POST":
        username = request.POST['username']
        password = request.POST['password']
        user = authenticate(request, username=username, password=password)

```

```

        if user is not None and not user.is_superuser:
            login(request, user)
            return redirect('user_panel') # replace with your user dashboard URL name
        else:
            messages.error(request, "Invalid credentials or not a user account.")

    return render(request, 'login_user.html')

def register_view(request):
    if request.method == "POST":
        username = request.POST['username']
        email = request.POST['email']
        password1 = request.POST['password1']
        password2 = request.POST['password2']

        if password1 != password2:
            messages.error(request, "Passwords do not match.")
        elif User.objects.filter(username=username).exists():
            messages.error(request, "Username already exists.")
        elif User.objects.filter(email=email).exists():
            messages.error(request, "Email already in use.")
        else:
            user = User.objects.create_user(username=username, email=email,
password=password1)
            messages.success(request, "Registration successful! You can now log in.")
            return redirect('login_user')

    return render(request, 'registration.html')

@login_required
@user_passes_test(lambda u: u.is_superuser)
def admin_panel(request):

```

```

databases = UploadedDatabase.objects.all().order_by('-uploaded_at')
recent_queries = UserQuery.objects.select_related('user').order_by('-timestamp')[:5]
result = None
query = ""
selected_db = None

```

```

if request.method == 'POST':
    db_id = request.POST.get('database')
    query = request.POST.get('query')
    selected_db = UploadedDatabase.objects.get(id=db_id)

    db_path = selected_db.file.path
    try:
        conn = sqlite3.connect(db_path)
        cursor = conn.cursor()

        cursor.execute(query)
        if query.strip().lower().startswith("select"):
            rows = cursor.fetchall()
            columns = [desc[0] for desc in cursor.description]
            result = {
                'columns': columns,
                'rows': rows
            }
        else:
            conn.commit()
            messages.success(request, "Query executed successfully.")
            result = {
                'columns': ['Status'],
                'rows': [["Query executed successfully."]]
            }

```

Optional: Save the query to UserQuery table for tracking (as admin query)

```

        if request.user.is_authenticated:
            UserQuery.objects.create(user=request.user, text=query)

    except Exception as e:
        messages.error(request, f'Error executing query: {e}')
    finally:
        conn.close()

    return render(request, 'admin_panel.html', {
        'databases': databases,
        'recent_queries': recent_queries,
        'result': result,
        'query': query,
        'selected_db': selected_db
    })

@login_required
def user_panel(request):
    """Displays the user interface for selecting a database and entering queries"""
    databases = UploadedDatabase.objects.all()
    return render(request, 'user_panel.html', {'databases': databases})

@login_required
@user_passes_test(is_admin)

def upload_database(request):
    if request.method == 'POST':
        uploaded_file = request.FILES.get('database_file')
        if uploaded_file:
            UploadedDatabase.objects.create(
                name=uploaded_file.name,
                file=uploaded_file,
                uploaded_by=request.user

```

```

    )
    messages.success(request, 'Database uploaded successfully!')
else:
    messages.error(request, 'No file selected.')

    return redirect('admin_panel') # Change to match your url name

    return redirect('admin_panel')
def get_schema(database_path):
    """Extracts schema information from an SQLite database using SQLAlchemy"""
    engine = create_engine(f'sqlite:/// {database_path}')
    with engine.connect() as conn:
        tables = conn.execute(text("SELECT name FROM sqlite_master WHERE
type='table';")).fetchall()

    schema = {}
    for table in tables:
        table_name = table[0]
        with engine.connect() as conn:
            columns = conn.execute(text(f'PRAGMA
table_info({table_name})')).fetchall()
            schema[table_name] = [col[1] for col in columns]

    return schema

@login_required
def fetch_schema(request):
    """Fetch schema & table preview"""
    database_id = request.GET.get('database_id')
    db_obj = UploadedDatabase.objects.get(id=database_id)
    schema = get_schema(db_obj.file.path)

    engine = create_engine(f'sqlite:/// {db_obj.file.path}')

```

```

table_previews = {}
for table in schema.keys():
    with engine.connect() as conn:
        preview = pd.read_sql(f"SELECT * FROM {table} LIMIT 5", conn)
        table_previews[table] = preview.to_html(classes="table table-striped",
index=False)

```

```

return JsonResponse({'schema': schema, 'table_previews': table_previews})

```

```

@login_required

```

```

def process_query(request):

```

```

    """Processes natural language queries into SQL using Gemini API"""

```

```

    if request.method == 'POST':

```

```

        database_id = request.POST.get('database_id')

```

```

        user_query = request.POST.get('user_query')

```

```

    if not database_id or not user_query:

```

```

        return JsonResponse({'error': 'Missing database ID or query'}, status=400)

```

```

    try:

```

```

        db_obj = UploadedDatabase.objects.get(id=database_id)

```

```

    except UploadedDatabase.DoesNotExist:

```

```

        return JsonResponse({'error': 'Database not found'}, status=404)

```

```

    db_path = db_obj.file.path

```

```

    schema = get_schema(db_path)

```

```

    # Generate schema text for LLM prompt

```

```

    schema_text = "Database Schema:\n"

```

```

    for table, columns in schema.items():

```

```

        schema_text += f"\nTable: {table}\n"

```

```

        for col in columns:

```

```

            schema_text += f"- {col}\n"

```



```

# System prompt ensuring safe and correct SQL generation
system_prompt = (
    "You are a precise SQL query generator that converts natural language to SQL.\n"
    "Follow these rules strictly:\n"
    "1. Use exact column names from the schema.\n"
    "2. Include JOIN conditions only when the query requires data from more than one table and only if it's necessary.\n"
    "3. Use optimised SQL query for performance.\n"
    "4. Add appropriate WHERE clauses for filtering.\n"
    "5. Use appropriate aggregation functions when necessary.\n"
    "6. Always alias complex columns for readability.\n"
    "7. Return ONLY the SQL query without any explanation or markdown."
)

# Construct final prompt
prompt = (
    f"{system_prompt}\n"
    f"Generate a SQL query for: \"{user_query}\" \n"
    f"Using this schema:\n{schema_text}\n"
    "Return ONLY the SQL query without any explanations or markdown."
)

try:
    response = client.models.generate_content(
        model='gemini-2.0-flash',
        contents=prompt,
    )
    sql_query = response.text.strip()

# Remove markdown formatting if present
if "`sql" in sql_query:

```

```

        match = re.search(r"``sql\n(.*)\n``", sql_query, re.DOTALL)
        if match:
            sql_query = match.group(1).strip()

        return JsonResponse({'sql_query': sql_query})
    except Exception as e:
        logger.error(f'Error processing query: {str(e)}')
        return JsonResponse({'error': f'Error processing query: {str(e)}'}, status=500)

    return JsonResponse({'error': 'Invalid request'}, status=400)

@login_required
def execute_query(request):
    """Executes an SQL query using SQLAlchemy"""
    if request.method == 'POST':
        database_id = request.POST.get('database_id')
        sql_query = request.POST.get('sql_query')

        if not database_id or not sql_query:
            return JsonResponse({'error': 'Missing database ID or SQL query'}, status=400)

        try:
            db_obj = UploadedDatabase.objects.get(id=database_id)
        except UploadedDatabase.DoesNotExist:
            return JsonResponse({'error': 'Database not found'}, status=404)

        db_path = db_obj.file.path
        engine = create_engine(f'sqlite:/// {db_path}')

        try:
            with engine.connect() as conn:
                if sql_query.strip().upper().startswith("SELECT"):
                    result = pd.read_sql(sql_query, conn)

```

```

        return JsonResponse({'results': result.to_html(classes="table table-bordered", index=False)})
    else:
        if request.user.is_superuser:
            with conn.begin():
                conn.execute(text(sql_query))
            return JsonResponse({'message': 'Query executed successfully!'})
        else:
            return JsonResponse({'error': 'Permission denied! Only admins can modify the database.'}, status=403)
    except Exception as e:
        return JsonResponse({'error': f'SQL execution error: {str(e)}'}, status=500)

return JsonResponse({'error': 'Invalid request'}, status=400)

class LogoutViaGet(LogoutView):
    def get(self, request, *args, **kwargs):
        return self.post(request, *args, **kwargs)

```

9.2 Settings.py

```

from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'django-insecure-zegd_@0^n-7yh*x$+v7n#usf=aj-@p(ksz93lm6j3eni+q5-%8'

DEBUG = True

ALLOWED_HOSTS = []

INSTALLED_APPS = [

```

```

'django.contrib.admin',
'django.contrib.auth',
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
'text2sql',
]

```

```

MIDDLEWARE = [
'django.middleware.security.SecurityMiddleware',
'django.contrib.sessions.middleware.SessionMiddleware',
'django.middleware.common.CommonMiddleware',
'django.middleware.csrf.CsrfViewMiddleware',
'django.contrib.auth.middleware.AuthenticationMiddleware',
'django.contrib.messages.middleware.MessageMiddleware',
'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

```

```

ROOT_URLCONF = 'AutoSQL.urls'

```

```

TEMPLATES = [
{
'BACKEND': 'django.template.backends.django.DjangoTemplates',
'DIRS': [],
'APP_DIRS': True,
'OPTIONS': {
'context_processors': [
'django.template.context_processors.debug',
'django.template.context_processors.request',
'django.contrib.auth.context_processors.auth',
'django.contrib.messages.context_processors.messages',
],
},
}
]

```

```

        },
    },
]

WSGI_APPLICATION = 'AutoSQL.wsgi.application'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

STATIC_URL = 'static/'

# Added manually

MEDIA_URL = '/media/'

```

```

MEDIA_ROOT = BASE_DIR / 'media'
LOGIN_REDIRECT_URL = '/text2sql/user-panel/'
LOGOUT_REDIRECT_URL = '/'
LOGIN_URL = '/'
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
# Google Gemini API Key
GEMINI_API_KEY = "YOUR_GEMINI_API_KEY"

```

9.3 Manage.py

```

import os
import sys

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'AutoSQL.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()

```

CHAPTER – 10

OUTPUT

SCREENS

CHAPTER 10

OUTPUT SCREENS

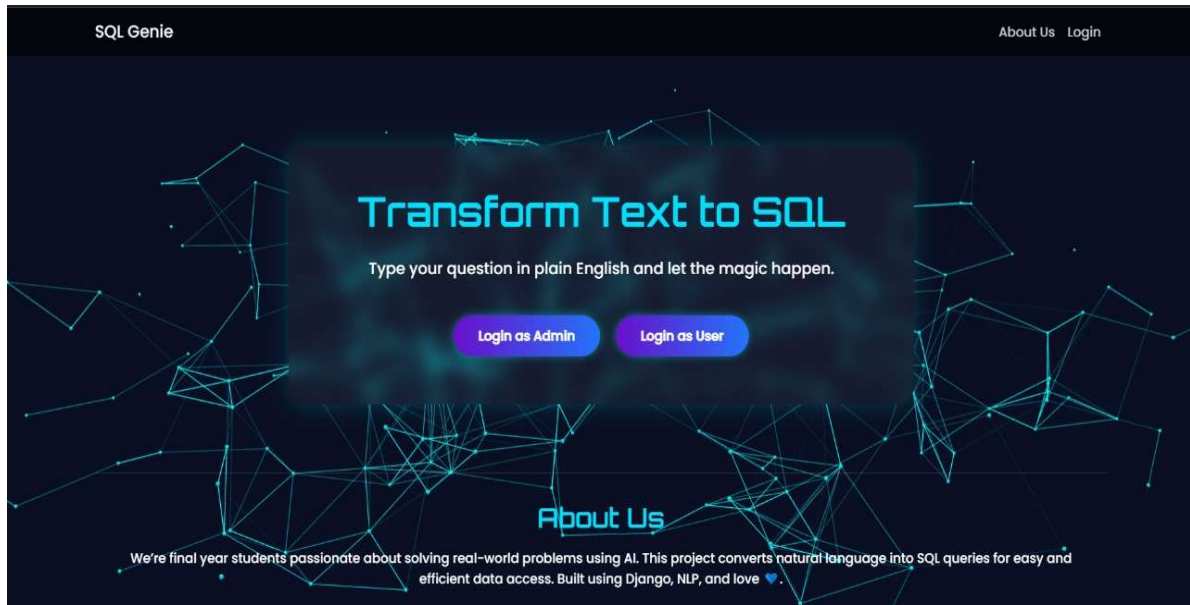


Figure 10.1: Home Page

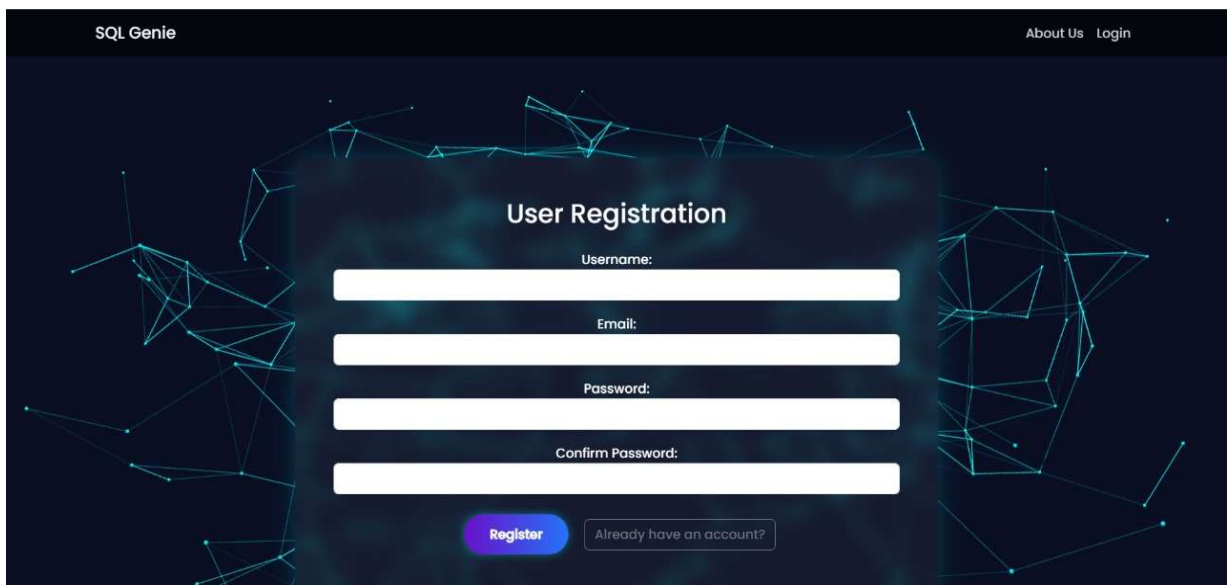


Figure 10.2: Registration Page

+

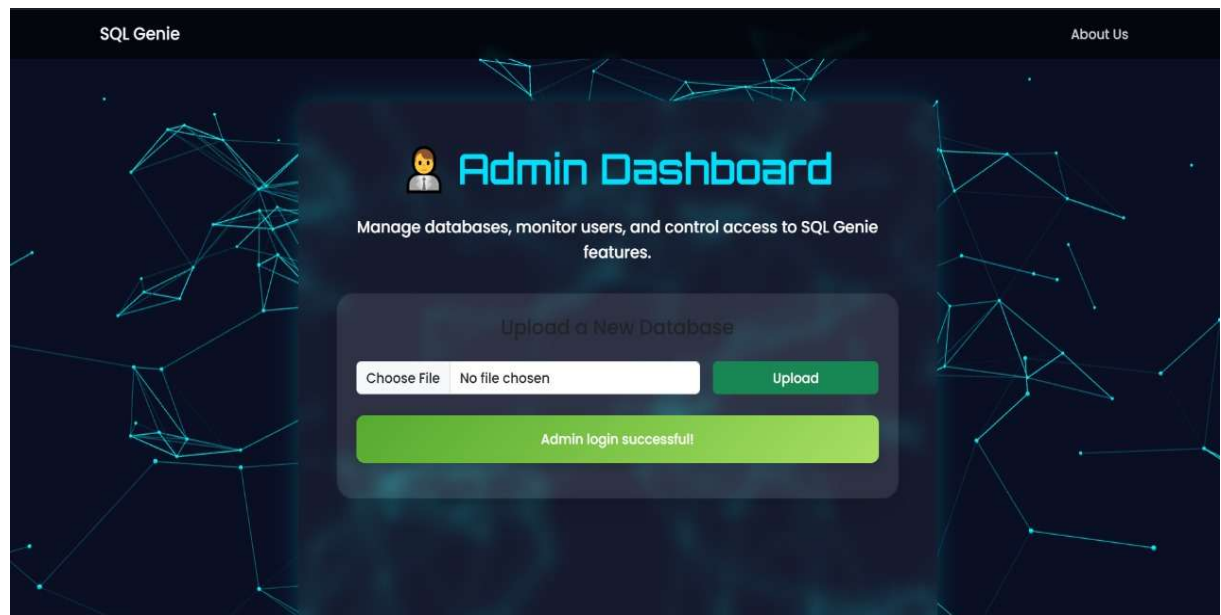


Figure 10.3: Admin Dashboard

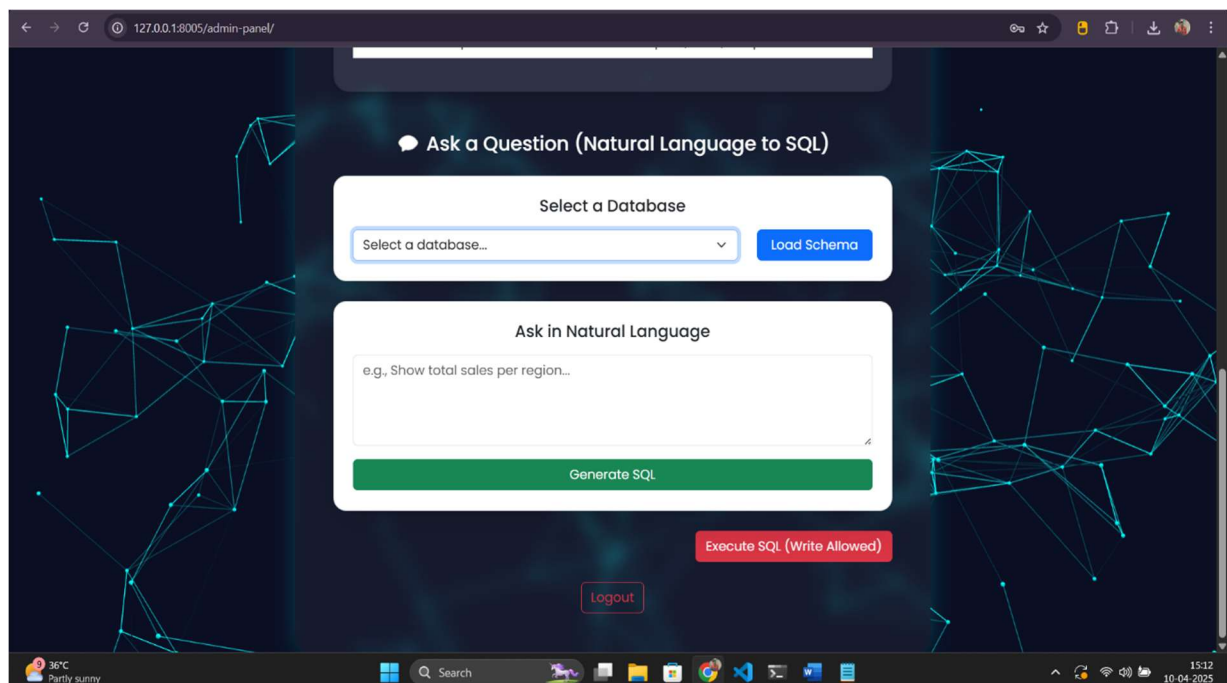


Figure 10.4: Execution Interface

CHAPTER – 11

FUTURE WORK

CHAPTER 11

FUTURE WORK

1. Limited Model Capabilities

The application uses Gemini 2.0 Flash, which, while lightweight and fast, does not offer the advanced contextual understanding and reasoning abilities of more powerful LLMs like Gemini Pro or GPT-4. Enhancement: Future versions can integrate more advanced models that better understand complex queries, multi-step reasoning, and ambiguous user inputs. Additionally, fine-tuning the model on SQL-specific prompts can improve accuracy.

2. Lack of Real-Time Database Feedback

Currently, the chatbot generates SQL queries without validating them against actual schema constraints like foreign keys, data types, or existing values. Enhancement: Add a schema-aware assistant module that checks query validity in real time and provides intelligent suggestions based on the specific structure and content of the connected database.

3. No Support for Advanced SQL Operations

The system mainly handles basic operations like SELECT, UPDATE, DELETE, and INSERT. It lacks support for joins, subqueries, transactions, views, and stored procedures.

Enhancement: Extend the prompt engineering and post-processing to support advanced SQL use cases. You could also maintain a query context to support multi-step conversations involving temporary table creation or nested queries.

4. Limited Error Handling and Feedback

Shortcoming: If a generated query fails or causes an error, the feedback to the user is minimal and non-informative. **Enhancement:** Improve error handling by capturing detailed database error messages and providing friendly explanations or auto-corrections. Integrate logs to show why a query failed and offer corrected alternatives.

5. Security Concerns

Shortcoming: The application currently lacks robust protection against SQL injection, data leaks, and unauthorized database access. **Enhancement:** Implement strict user role-based access control (RBAC), input sanitization, query validation, and query rate-limiting. Also, consider adding query approval workflows for non-read-only users.

6. No Natural Language Feedback for Query Results

Shortcoming: While the app returns tabular results, it doesn't summarize or explain what the results mean in plain English. **Enhancement:** Introduce a response summarizer module using the LLM that describes the output in user-friendly language, making it more accessible to non-technical users.

7. No Learning from User Interactions

Shortcoming: The system does not adapt to frequent user queries, corrections, or preferences.

Enhancement: Add a feedback mechanism that learns from user interactions over time, either through explicit ratings or implicit usage patterns, and tunes prompts or models accordingly.

8. Lack of Mobile Responsiveness and UX Optimization

Shortcoming: While the web interface works, it isn't optimized for mobile users or those with accessibility needs. **Enhancement:** Redesign the UI with responsive, accessible layouts and optionally provide a mobile app version for enhanced convenience.

9. Lack of Multi-Database Support

Shortcoming: The system can currently operate on one database per session.

Enhancement: Add support for switching across multiple databases dynamically, each with its own schema, permissions, and access logs.

10. No Logging or Analytics of Usage Patterns

Shortcoming: The application doesn't currently log user queries or system usage.

Enhancement: Integrate a logging system for query histories, usage analytics, and model performance metrics, which can later guide improvements and debugging.

APPENDIX-1

GLOSSARY OF TERMS

(In alphabetical order)

A

- API** Application Programming Interface - A set of protocols and tools that allow different software applications to communicate with each other. In this project, the Gemini API is used to convert natural language into SQL queries.
- Authentication** A process that verifies the identity of a user. The Django framework includes built-in authentication to manage access control for superusers and staff.
- Authorization** Determines what a user is allowed to do after authentication. In this system, users can have limited or full database access depending on their assigned roles.

B

- Backend** The server-side logic of a web application that includes APIs, business logic, and database operations.

D

- Django** A high-level Python web framework that promotes rapid development and clean, pragmatic design. It is used as the backend framework for this project.
- Database Schema** The structure of a database defined in terms of tables, columns, and their relationships. This schema is provided to the LLM for generating contextually correct SQL queries.

F

- Front-End** The part of a web application visible to the user, usually consisting of HTML, CSS, and JavaScript. It communicates with the Django backend to allow query input and result display.

G

Gemini-2.0 Flash A lightweight and fast variant of Google’s Gemini large language model (LLM), used in this project for transforming user queries in natural language to SQL format via API.

H

HTML The standard language used to create and structure content on the web. It defines the elements and layout of web pages through tags and attributes.

I

IDE A software application that provides tools for software development, including a code editor, debugger, and compiler. It simplifies the process of writing, testing, and debugging code.

L

LLM Large Language Model - An artificial intelligence model trained on large datasets of human language to understand and generate text. In this project, it is used for interpreting and converting text into SQL queries.

N

Natural Language Processing (NLP) A subfield of artificial intelligence that focuses on the interaction between computers and human language, used to convert natural language into machine-readable commands.

Natural Language Query A user input written in everyday human language (e.g., “Show me all users registered in March”), which is processed by the LLM to generate a valid SQL query.

O

ORM (Object Relational Mapping) A programming technique used to interact with the database using Python objects instead of writing raw SQL.

P

Permissions A mechanism to restrict or allow certain actions in the application. Superusers can assign query execution permissions (like SELECT-only or full access) to staff.

PostgreSQL / SQLite / Relational database management systems (RDBMS) used to store, manage, and retrieve data. SQLite is used for development while PostgreSQL may be used in production.

Q

Query Execution The process of running a SQL statement against a selected database and returning the results or modifying the data, depending on the type of query.

Query Logging The process of storing all queries entered and processed by the application for audit, analysis, or debugging purposes.

R

Role-Based Access Control (RBAC) A security principle in which users are assigned roles, and each role is granted specific permissions. This ensures that only authorized users can access certain features.

S

SQL (Structured Query Language) A standard language used to manage and manipulate relational databases. The LLM generates SQL queries from plain English input in this project.

Superuser An administrative user in Django with full permissions, responsible for creating databases, managing users, and assigning roles in the application.

Session Management The method of maintaining the state of users across different requests in a web application.

U

UI The space where interactions between humans and machines occur. It includes the design and layout of elements like buttons, menus, and screens that allow users to interact with software or hardware in a way that is intuitive and efficient.

V

VS CODE An open-source code editor developed by Microsoft which supports a wide range of programming languages and features such as debugging, syntax highlighting, version control integration, etc.

W

Web Application

A software application that runs in a web browser and can be accessed via the internet or an intranet. This project is delivered as a Django-powered web application.

REFERENCES

JOURNALS / RESEARCH PAPERS

1. **Ali Mohammadjafari, Anthony S. Maida, and Raju Gottumukkala**, "From Natural Language to SQL: Review of LLM-based Text-to-SQL Systems," *arXiv preprint arXiv:2410.01066*, 2023. [Online]. Available: <https://arxiv.org/abs/2410.01066>
2. **Dawei Gao et al.**, "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," *arXiv preprint arXiv:2308.15363*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.15363>
3. **Zijin Hong et al.**, "Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL," *arXiv preprint arXiv:2406.08426*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.08426>
4. **Zihui Gu et al.**, "Interleaving Pre-Trained Language Models and Large Language Models for Zero-Shot NL2SQL Generation," *arXiv preprint arXiv:2306.08891*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.08891>
5. **Shayan Talaei et al.**, "CHESS: Contextual Harnessing for Efficient SQL Synthesis," *arXiv preprint arXiv:2405.16755*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.16755>.

WEBSITES

6. **BIRD (BIg Bench for LaRge-scale Database Grounded Text-to-SQL Evaluation)** Benchmark. <https://bird-bench.github.io/>

PROJECT SUMMARY

About Project

Title of the project	Django Based Text-To-Sql Execution Interface
Semester	8 th
Members	1. Mahak Mirza 2. Sparsh Sahu 3. Eashan Tiwari
Team Leader	Sparsh Sahu
Describe role of every member in the project	Sparsh Sahu: LLM integration + Backend (Django) + API handling Mahak Mirza: Frontend (HTML, CSS, JS) + UI design Eashan Tiwari: Database setup + Permissions + Backend support
What is the motivation for selecting this project?	The motivation behind this project stems from the need to simplify the way non-technical users interact with relational databases. Writing SQL queries requires technical knowledge, which is often a barrier in data-driven environments. This project bridges that gap by integrating a large language model (LLM) with a Django web application, enabling users to input natural language queries and receive accurate SQL query results without prior knowledge of SQL syntax. It promotes accessibility, productivity, and error-free database querying in organizations.
Project Type (Desktop Application, Web Application, Mobile App, Web)	Web Application

Tools & Technologies

Programming language used	Python
Compiler used (with version)	NA
IDE used (with version)	Microsoft Visual Studios Code (1.51.1)
Front End Technologies (With version, wherever Applicable)	HTML, CSS & JavaScript

Back End Technologies (with version, wherever applicable)	Django (Python) v 5.1.6
Database used (with version)	SQLite (for development), PostgreSQL (for production-ready deployment)

Software Design& Coding

Is prototype of the software developed?	Yes
SDLC model followed (Waterfall, Agile, Spiral etc.)	Agile Methodology
Why above SDLC model is followed?	The Agile model supports frequent iteration and collaboration, which was necessary because the project involved continuous testing with the LLM and frequent UI/backend changes based on evolving requirements and integration feedback.
Justify that the SDLC model mentioned above is followed in the project.	The functionality of the web application kept evolving due to changes in permissions, API integrations, and user interface components. These iterative improvements aligned with Agile's sprint-based, flexible approach.
Software Design approach followed (Functional or Object-oriented)	Functional oriented approach
Name the diagrams developed (According to the Design approach followed)	Use Case Diagram, Data Flow Diagram (DFD), Class Diagram
In case Object Oriented approach is followed, which of the OOPS principles are covered in design?	NA

No. of Tiers (example 3-tier)	3-tier architecture
Total no. of front-end pages	3 (Login, Query Interface, Query Result Page)
Total no. of tables in database	2 (Users, Query Logs)
Database is in which Normal Form?	Third Normal Form (3NF)
Are the entries in database encrypted?	User passwords are hashed using Django's default encryption
Front end validations applied (Yes / No)	Yes
Session management done (in case of web applications)	Yes (Django's built-in session management)
Is application browser compatible (in case of web applications)	Yes
Exception handling done (Yes / No)	Yes
Commenting done in code (Yes / No)	Yes
Naming convention followed (Yes / No)	Yes
What difficulties faced during deployment of project?	• Understanding the limitations of a lightweight LLM (Gemini 2.0 Flash) while ensuring reliable SQL conversion. • Managing permissions and query restrictions to prevent unauthorized access to sensitive database actions. • Integrating frontend, backend, and Gemini API in a modular and secure way. • Building a flexible architecture to handle multiple databases and multiple users
Total no. of Use-cases	4
Give titles of Use-cases	1. Natural Language Query Input 2. SQL Query Generation via Gemini 3. Query Execution on Selected Database 4. Display and Logging of Results

Project Requirements

MVC architecture followed (Yes / No)	Yes
If yes, write the name of MVC architecture followed (MVC-1, MVC-2)	MVC-2
Design Pattern used (Yes / No)	No
If yes, write the name of Design Pattern used	No
Interface type (CLI / GUI)	GUI
No. of Actors	2
Name of Actors	Superuser, Staff/User
Total no. of Functional Requirements	6
List few important non-Functional Requirements	Accuracy, Maintainability, Responsiveness, Security, Compatibility

Testing

Which testing is performed? (Manual or Automation)	Manual
Is Beta testing done for this project?	No

Write project narrative covering above mentioned points

The project titled “*Design and Development of a Natural Language to SQL Web Interface using LLM*” addresses the challenge of bridging human language and database communication. It provides a simple and intuitive web interface where users can type questions in natural English and receive real-time SQL-generated results. Using Gemini 2.0 Flash, the project translates these inputs into valid SQL statements, which are then executed on a selected database. Built using Django and designed with clean frontend principles, the system also supports multi-user login, role-based permissions, and supports query logging. This system enables non-technical users to perform complex database operations efficiently, making enterprise data more accessible and reducing reliance on database administrators. It offers a scalable, secure, and intelligent platform for real-time database interaction through natural language.

Mahak Mirza
0187AD211020

Sparsh Sahu
0187AD211039

Eashan Tiwari
0187AD2110015

Guide Signature
Dr. Vasima Khan